Python Quantum Logic Visualiser:

Project diary

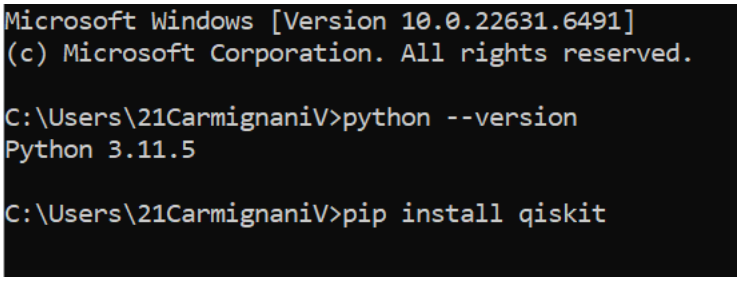
3/2/26 - week 1 – Today I have begun by establishing and confirming my main idea – which is:

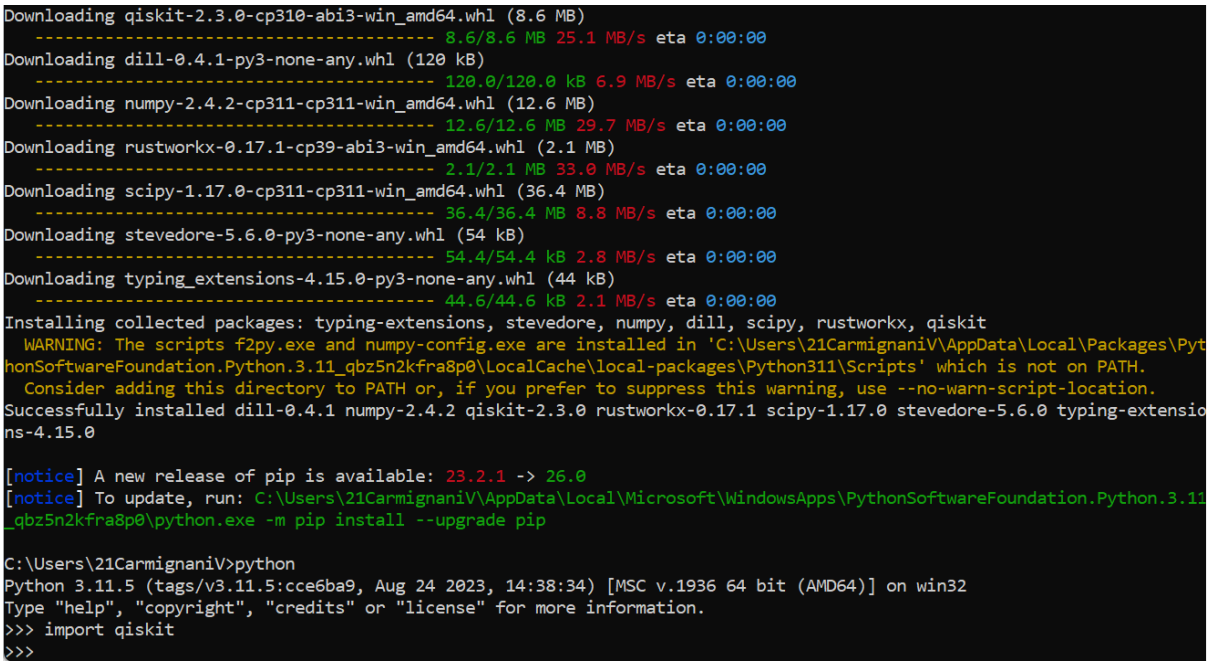
“I am exploring a program that simplifies the concept of quantum mathematics to make it more digestible for me and others who want to understand the basics of quantum computing. My project focuses on building curiosity for quantum computation through simulation, rather than assuming understanding.”

Firstly, I recapped that a qubit is different to a classical bit. This is evident as a classical bit can be either a 0 or a 1. Whereas a qubit can be 0, 1 or a combination (superposition) since a qubit can be in multiple states at once until measured. This is why quantum computers work with probabilities not fixed answers.

I then opened python just to begin my initial round of exploration of my tools:

1. I had to import Qiskit (IBM’s quantum library) on command prompt(cmd).





2. Then, in IDLE, I imported the Qiskit’s tool for building quantum circuits (designing the experiment) and AerSimulator, which simulates a quantum computer on a normal one (this is for running the experiment itself).

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
```

When I ran this it came up with an error message saying that it could not recognise the module “qiskit”. Therefore I went on cmd again and imported it again making sure to import it into the python I am working on and testing it on both sides.

```
C:\Users\21CarmignaniV>python -m pip install qiskit
Requirement already satisfied: qiskit in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.python.3
.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (2.3.0)
Requirement already satisfied: rustworkx>=0.15.0 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundati
on.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit) (0.17.1)
Requirement already satisfied: numpy<3,>=1.17 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.
python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit) (2.4.2)
Requirement already satisfied: scipy>=1.5 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.pyth
on.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit) (1.17.0)
Requirement already satisfied: dill>=0.3 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.pytho
n.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit) (0.4.1)
Requirement already satisfied: stevedore>=3.0.0 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundatio
n.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit) (5.6.0)
Requirement already satisfied: typing-extensions in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundati
on.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit) (4.15.0)

[notice] A new release of pip is available: 23.2.1 -> 26.0
[notice] To update, run: C:\Users\21CarmignaniV\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11
_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

C:\Users\21CarmignaniV>python
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import qiskit
>>> print("Qiskit works")
Qiskit works
>>> exit
```

```
Python 3.11.5 (tags/v3.11.5:cce6ba
AMD64)] on win32
Type "help", "copyright", "credits'
import qiskit
print(
print(qiskit.__version__)
2.3.0
```

3. Now that the module is working, I began modelling a qubit in the simulation by placing it into a state of equal probability, demonstrating quantum superposition.

```
from qiskit import QuantumCircuit #importing the tool that helps me design the experiment.
qc=QuantumCircuit(1)#creates a quantum circuit with 1 qubit(in its ground state).
qc.h(0)#hadamard gate - puts the qubit into a superposition (0 and 1 at the same time)
print(qc)
```

This was the result (the circuit drawn):

```
q: ┤ H ├
```

12/02/2026 – week 2 – Today I am aiming to demonstrate a concept very fundamental to quantum computing. It is that **a qubit does not give a value until you measure it and measurement is random but follows probability**.

Today I aim to code a simulation that will:

- Measure the qubit.

- Run the experiment several times.

- Displays a 50/50 distribution.

First I began with a similar setup as last time only I'm utilising the simulator too this time:

```
*week2sim.py - C:/Users/21CarmignaniV/OneDriv
File  Edit  Format  Run  Options  Window  He
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
```

```
week2sim.py - C:/Users/21CarmignaniV/OneDrive - Lancaster Girls' Grammar School/personal/arkwright/week2sim.py (3.11.5)
File  Edit  Format  Run  Options  Window  Help
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
qc=QuantumCircuit(1,1)#1 qubit and 1 classical bit where the answer is stored.
qc.h(0)#puts the qubit into a superposition - it is now 50% |0) and 50% |1)
qc.measure(0,0)#this is telling the program to take qubit 0 and store the result in classical bit 0.
sim=AerSimulator()
job=sim.run(qc, shots=1000)
result=job.result()
counts=result.get_counts()
print(counts)
#this part of the code runs the simulation many times.
```

```
Traceback (most recent call last):
  File "C:/Users/21CarmignaniV/OneDrive - Lancaster Girls' Grammar School/person
al/arkwright/week2sim.py", line 2, in <module>
    from qiskit_aer import AerSimulator
ModuleNotFoundError: No module named 'qiskit_aer'
>>>
```

I tried to run this commented code but was again having problems with the modules being installed so I went on cmd again to install it and test it as shown below.

```
Microsoft Windows [Version 10.0.22631.6649]
(c) Microsoft Corporation. All rights reserved.

C:\Users\21CarmignaniV>python -m pip install qiskit-aer
Collecting qiskit-aer
  Obtaining dependency information for qiskit-aer from https://files.pythonhosted.org/packages/82/8f/ac0a55a6fb539355e9e629f42f03032f9e93acce2390a87a7ab8c56764f7/qiskit_aer-0.17.2-cp311-cp311-win_amd64.whl.metadata
  Downloading qiskit_aer-0.17.2-cp311-cp311-win_amd64.whl.metadata (8.5 kB)
Requirement already satisfied: qiskit>=1.1.0 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit-aer) (2.3.0)
Requirement already satisfied: numpy>=1.16.3 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit-aer) (2.4.2)
Requirement already satisfied: scipy>=1.0 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit-aer) (1.17.0)
Collecting psutil>=5 (from qiskit-aer)
  Obtaining dependency information for psutil>=5 from https://files.pythonhosted.org/packages/b4/90/e2159492b5426be0c1fef7acba807a03511f97c5f86b3caeda6ad92351a7/psutil-7.2.2-cp37-abi3-win_amd64.whl.metadata
  Downloading psutil-7.2.2-cp37-abi3-win_amd64.whl.metadata (22 kB)
Collecting python-dateutil>=2.8.0 (from qiskit-aer)
  Obtaining dependency information for python-dateutil>=2.8.0 from https://files.pythonhosted.org/packages/ec/57/56b9bcc3c9c6a792fcbaf139543cee77261f3651ca9da0c93f5c1221264b/python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl.metadata (8.4 kB)
Collecting six>=1.5 (from python-dateutil>=2.8.0->qiskit-aer)
  Obtaining dependency information for six>=1.5 from https://files.pythonhosted.org/packages/b7/ce/149a00dd41f10bc29e5921b496af8b574d8413afcd5e30dfa0ed46c2cc5e/six-1.17.0-py2.py3-none-any.whl.metadata
  Downloading six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: rustworkx>=0.15.0 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit>=1.1.0->qiskit-aer) (0.17.1)
Requirement already satisfied: dill>=0.3 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit>=1.1.0->qiskit-aer) (0.4.1)
Requirement already satisfied: stevedore>=3.0.0 in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit>=1.1.0->qiskit-aer) (5.6.0)
Requirement already satisfied: typing-extensions in c:\users\21carmignaniv\appdata\local\packages\pythonsoftwarefoundation.python.3.11_qbz5n2kfra8p0\localcache\local-packages\python311\site-packages (from qiskit>=1.1.0->qiskit-aer) (4.15.0)
Downloading qiskit_aer-0.17.2-cp311-cp311-win_amd64.whl (9.6 MB)
   ---------------------------------------- 9.6/9.6 MB 20.4 MB/s eta 0:00:00
Downloading psutil-7.2.2-cp37-abi3-win_amd64.whl (137 kB)
   ---------------------------------------- 137.7/137.7 kB ? eta 0:00:00
Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
   ---------------------------------------- 229.9/229.9 kB 13.7 MB/s eta 0:00:00
Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, psutil, python-dateutil, qiskit-aer
Successfully installed psutil-7.2.2 python-dateutil-2.9.0.post0 qiskit-aer-0.17.2 six-1.17.0

[notice] A new release of pip is available: 23.2.1 -> 26.0.1
[notice] To update, run: C:\Users\21CarmignaniV\AppData\Local\Microsoft\WindowsApps\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\python.exe -m pip install --upgrade pip

C:\Users\21CarmignaniV>python
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from qiskit_aer import AerSimulator
>>> print("simulator ready.")
simulator ready.
>>>
```

The code now worked ideally as shown below.

The numbers did not match, and I did not get a fixed answer. Instead, the qubit collapsed to 0, sometimes to 1 and over many runs settled to approximately 50/50.

This is quantum probability, not just unpredictable randomness – and these probabilities come from the wavefunction created by the H gate.

In summary: Today I implemented measurement on a superposition state. Repeating the circuit 1000 times produced an approximately equal distribution of 0 and 1, demonstrating probabilistic collapse of the qubit state upon measurement.

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
qc=QuantumCircuit(1,1)#1 qubit and 1 classical bit where the answer is stored.
qc.h(0)#puts the qubit into a superposition - it is now 50% |0) and 50% |1)
qc.measure(0,0)#this is telling the program to take qubit 0 and store the result in classical bit 0.
sim=AerSimulator()
job=sim.run(qc, shots=1000)
result=job.result()
counts=result.get_counts()
print(counts)
#this part of the code runs the simulation many times.
```

```
{'0': 504, '1': 496}
```

24/02/2026 – week 3 – Today I am planning to move closer to ZX and closer to the core of my project. And before I start using libraries, I want to understand the difference between circuits and ZX as the latter are graphs rather than circuits.

I did this by first playing around with a list of operations:

```
circuit=[
    ("h",0),
    ("CNOT",0, 1),
    ("Z", 1)
]
print(circuit)
```

```
[('h', 0), ('CNOT', 0, 1), ('Z', 1)]
>>>
```

This shows that they exist in python memory.

Now I want to start exploring three things within ZX diagrams:

- Nodes (spiders)
- Edges (wires)
- Types (Z or X)

First, I am going to create a basic 'spider' class, again just becoming familiar with the graphic structure.

```
class Spider:
    def SpiderTry(self, spider_type, phase=0):
        self.type=spider_type #Z or X - building the language layer.
        self.phase=phase
        self.connections=[]
```

I added this section to my code. Again, at this point this is just the language layer, somewhat like the basic vocabulary that my program needs to be able to represent quantum circuits in pictorial form. Here, I am defining what a spider is, what information it stores and how it connects to others. Evidently, this is one of the foundational code structures that define how my program understands quantum picturalism

Next, I am going to attempt to convert a gate into a spider but as a simplified idea to begin with as I am not doing full mathematical correctness yet.

For now, I am going to consider:

- H gate → X spider (again simplified idea)

- Z gate → Z spider
- CNOT → 2 spider connected

This is what the code currently looks like:

```
circuit=[
    ("h",0),
    ("CNOT",0, 1),
    ("Z", 1)
]
print(circuit)
class Spider:
    def SpiderTry(self, spider_type, phase=0):#first function, passing self, spider_type, phase=0 as parameters.
        self.type=spider_type #Z or X - building the language layer.
        self.phase=phase
        self.connections=[]
def GateToSpider(gate):#second function and passing gate as a function.
    if gate[0]=="H":
        return [Spider("X")]
    elif gate[0]=="Z":
```

Right now my function only handles single qubit gates like (“H”, 0) (“Z”,1), but CNOT is different because it involves two qubits as it needs two spiders connected.

In ZX calculus CNOT is approximately equal to a Z spider (control) and an X spider (target) connected by a wire. So conceptually still only modelling the structure:

Z(control)---X(target)

This means that instead of returning just one spider sometimes it will return two.

```
circuit=[
    ("h",0),
    ("CNOT",0, 1),
    ("Z", 1)
]
print(circuit)
class Spider:
    def SpiderTry(self, spider_type, phase=0):#first function, passing self, spider_type, phase=0 as parameters.
        self.type=spider_type #Z or X - building the language layer.
        self.phase=phase
        self.connections=[]
def GateToSpider(gate):#second function and passing gate as a function.
    if gate[0]=="H":
        return [Spider("X")]
    elif gate[0]=="Z":
        return [Spider("Z")]
    elif gate[0]=="CNOT":
        control=Spider("Z")
        target=Spider("X")
        #then connect the two
        control.connections.append(target)
        target.connections.append(control)
        return [control,target]#now the function returns a list of spiders, not just one.
```

When I ran the code, this error message came up:

```
[('h', 0), ('CNOT', 0, 1), ('Z', 1)]
Traceback (most recent call last):
  File "C:/Users/21CarmignaniV/OneDrive - Lancaster Girls' Grammar School/personal/arkwright/week3operations.py", line 24, in <module>
    CNOTtest=GateToSpider(("CNOT",0,1))
  File "C:/Users/21CarmignaniV/OneDrive - Lancaster Girls' Grammar School/personal/arkwright/week3operations.py", line 18, in GateToSpider
    control=Spider("Z")
TypeError: Spider() takes no arguments
```

I then realised that spider cannot be used in that context as the Spider class doesn't have an initiation that accepts parameters. The __init__ is necessary to receive and store the data so Spider() can be used.

This is the fixed and successful code:

```
circuit=[
    ("h",0),
    ("CNOT",0,1),
    ("Z",1)
]
print(circuit)
class Spider:
    def __init__(self, spider_type, phase=0):#initialises the Spider - this is necessary for it to work.
        self.type=spider_type #Z or X - building the language layer.
        self.phase=phase
        self.connections=[]
def GateToSpider(gate):#second function and passing gate as a function.
    if gate[0]=="H":
        return [Spider("X")]
    elif gate[0]=="Z":
        return [Spider("Z")]
    elif gate[0]=="CNOT":
        control=Spider("Z")
        target=Spider("X")
        #then connect the two
        control.connections.append(target)
        target.connections.append(control)
        return [control,target]#now the function returns a list of spiders, not just one.
CNOTtest=GateToSpider(("CNOT",0,1))
for spider in CNOTtest:#testingcode
    print(spider.type,"connected to",
          [s.type for s in spider.connections])
```

And the results:

```
[('h', 0), ('CNOT', 0, 1), ('Z', 1)]
Z connected to ['X']
X connected to ['Z']
>>>
```

03/03/26 – week 4 – Today I am aiming to write the final code for converting circuits to ZX style diagrams. I began by creating a function for this; however, I encountered this error:

```
[('h', 0), ('CNOT', 0, 1), ('Z', 1)]
Z connected to ['X']
X connected to ['Z']
Traceback (most recent call last):
  File "C:/Users/21CarmignaniV/OneDrive - Lancaster Girls' Grammar School
al/arkwright/week3operations.py", line 31, in <module>
    zxgraph=ZXgraphbuild(circuit)
  File "C:/Users/21CarmignaniV/OneDrive - Lancaster Girls' Grammar School
al/arkwright/week3operations.py", line 29, in ZXgraphbuild
    graph.extend(spiders)
TypeError: 'NoneType' object is not iterable
```

```
circuit=[("h",0),("CNOT",0,1),("Z",1)]
print(circuit)

class Spider:
    def __init__(self, spider_type, phase=0):#initialises the Spider - this i
        self.type=spider_type #Z or X - building the language layer.
        self.phase=phase
        self.connections=[]
def GateToSpider(gate):#second function and passing gate as a function.
    if gate[0]=="H":
        return [Spider("X")]
    elif gate[0]=="Z":
        return [Spider("Z")]
    elif gate[0]=="CNOT":
        control=Spider("Z")
        target=Spider("X")
        #then connect the two
        control.connections.append(target)
        target.connections.append(control)
        return [control,target]#now the function returns a list of spiders, r
    else:
        return []
CNOTtest=GateToSpider(("CNOT",0,1))
for spider in CNOTtest:#testingcode
    print(spider.type,"connected to",
          [s.type for s in spider.connections])
def ZXgraphbuild(circuit):#made a function
    graph=[]
    for gate in circuit:
        spiders=GateToSpider(gate)
        graph.extend(spiders)
    return graph
zxgraph=ZXgraphbuild(circuit)
```

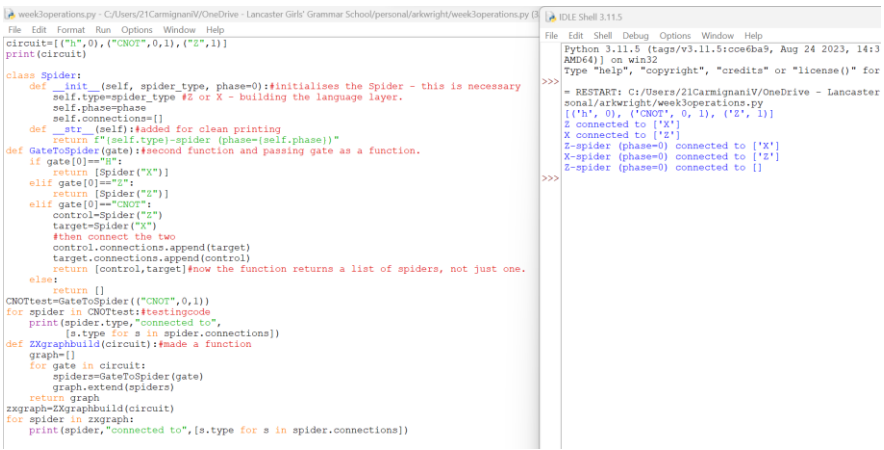
I wasn't sure why it was doing this possibly because spiders is None and I'm trying to extend() using it but None isn't a list so I added an else statement that I think will stop the program from crashing when something doesn't match.

When I then ran it the error message did not come up.

Next, inside my Spider class, I am going to ensure a clear print representation.

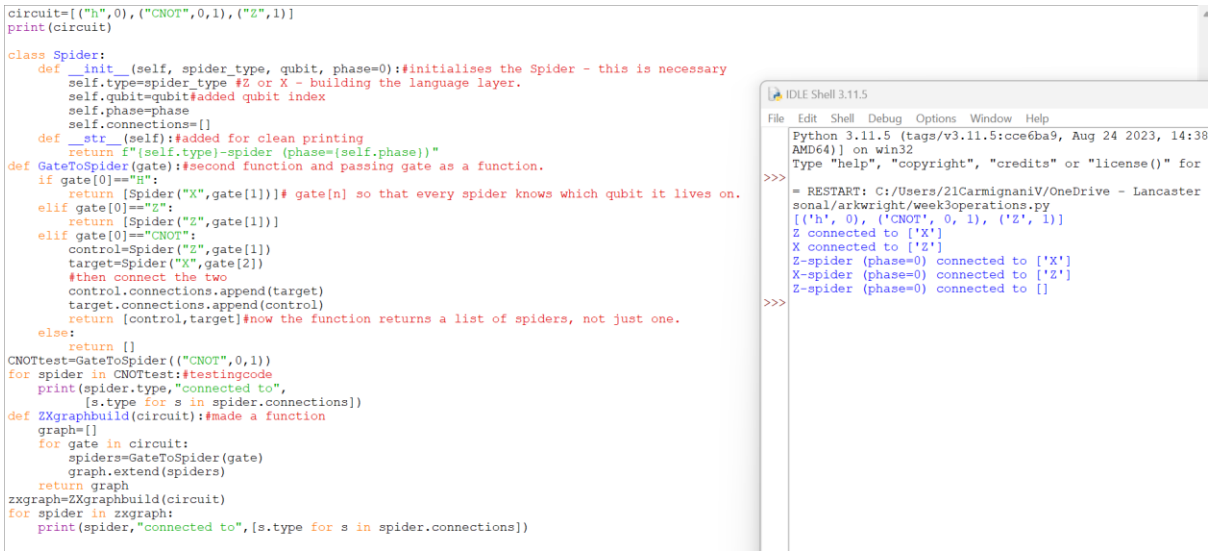
```
class Spider:
    def __init__(self, spider_type, phase=0):#initialises the Spider - this i
        self.type=spider_type #Z or X - building the language layer.
        self.phase=phase
        self.connections=[]
    def __str__(self):#added for clean printing
        return f"{self.type}-spider (phase={self.phase})"
```

Now I am going to print the full graph structure:



This result makes sense as I corresponds exactly to my circuit. The H gate produced an X spider, the CNOT produced a connected Z-X pair and Z produced a single Z spider with no connections so an empty list is correct.

Next, I am going to add qubit tracking by modifying Spider to include qubit index. SO that when creating spiders the model can understand wires.



```
circuit=[("h",0),("CNOT",0,1),("Z",1)]
print(circuit)

class Spider:
    def __init__(self, spider_type, qubit, phase=0):#initialises the Spider - this is necessary
        self.type=spider_type #Z or X - building the language layer.
        self.qubit=qubit#added qubit index
        self.phase=phase
        self.connections=[]
    def __str__(self):#added for clean printing
        return f"{self.type}-spider (phase={self.phase})"
def GateToSpider(gate):#second function and passing gate as a function.
    if gate[0]=="H":
        return [Spider("X",gate[1])]# gate[n] so that every spider knows which qubit it lives on.
    elif gate[0]=="Z":
        return [Spider("Z",gate[1])]
    elif gate[0]=="CNOT":
        control=Spider("Z",gate[1])
        target=Spider("X",gate[2])
        #then connect the two
        control.connections.append(target)
        target.connections.append(control)
        return [control,target]#now the function returns a list of spiders, not just one.
    else:
        return []
CNOTtest=GateToSpider(("CNOT",0,1))
for spider in CNOTtest:#testingcode
    print(spider.type,"connected to",
          [s.type for s in spider.connections])
def ZXgraphbuild(circuit):#made a function
    graph=[]
    for gate in circuit:
        spiders=GateToSpider(gate)
        graph.extend(spiders)
    return graph
zxgraph=ZXgraphbuild(circuit)
for spider in zxgraph:
    print(spider,"connected to",[s.type for s in spider.connections])
```

```
IDLE Shell 3.11.5
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for
>>>
= RESTART: C:/Users/21CarmignaniV/OneDrive - Lancaster
sonal/arkwright/week3operations.py
[('h', 0), ('CNOT', 0, 1), ('Z', 1)]
Z connected to ['X']
X connected to ['Z']
Z-spider (phase=0) connected to ['X']
X-spider (phase=0) connected to ['Z']
Z-spider (phase=0) connected to []
>>>
```

I would like to also attempt a visual upgrade so that the output is more suited to quantum picturalism and easier to read.

```
def __str__(self):#added for clean printing
    return f"{self.type}-spider (q{self.qubit},phase={self.phase})"#makes output more readable
```

```
[('h', 0), ('CNOT', 0, 1), ('Z', 1)]
Z connected to ['X']
X connected to ['Z']
Z-spider (q0,phase=0) connected to ['X']
X-spider (q1,phase=0) connected to ['Z']
Z-spider (q1,phase=0) connected to []
```

Now to make the connections so qubits too I tweaked this piece of code:

```
print(spider,"connected to",[f"{s.type}(q{s.qubit})" for s in spider.connections])
```

To get this output:

```
[('h', 0), ('CNOT', 0, 1), ('Z', 1)]
Z connected to ['X']
X connected to ['Z']
Z-spider (q0,phase=0) connected to ['X(q1)']
X-spider (q1,phase=0) connected to ['Z(q0)']
Z-spider (q1,phase=0) connected to []
```

Right now my Z gate creates a Z-spider, on a specific qubit with default phase=0. However, In ZX-calculus , Z-spiders can carry a phase , mathematically it can represent:

- Identity (phase=0)
- Pauli-Z (phase=π)
- S gate (phase=π/2)
- T gate (phase=π/4)
- Any rotation around Z.

So if I add phase then my gate tuple will mean Z gate on qubit 1 with phase π.

```
import math
circuit=[("h",0),("CNOT",0,1),("Z",1,math.pi/2)]######
print(circuit)
```

```
elif gate[0]=="Z":
    if len(gate)==3:#checks if user included a phase - if ys use it if no default to 0.#####
        return [Spider("Z",gate[1],gate[2])]####
    else:####
        return[Spider("Z",gate[1])]#####
```

So we expect:

1. X-spider on q0(from H)
2. Z-spider on q0 connected to X-spider on q1 (CNOT control)
3. X-spider on q1 connected to Z-spider on q0 (CNOT target)
4. Z-spider on q1 with phase π/2.

```
[('h', 0), ('CNOT', 0, 1), ('Z', 1, 1.5707963267948966)]
Z connected to ['X']
X connected to ['Z']
Z-spider (q0,phase=0) connected to ['X(q1)']
X-spider (q1,phase=0) connected to ['Z(q0)']
Z-spider (q1,phase=1.5707963267948966) connected to []
```

What the output shows:

1.5707963267948966... = π/2 – this is mathematically correct so the phase is being stored properly.

Spider Fusion

A rule in ZX-calculus is that if two connected spiders are the same type i.e. Z-Z or X-X or on the same qubit, they can fuse into one spider and their phases add. I'm going to implement some code to model this:

```
def fuse(spider1, spider2):
    if spider1.type==spider2.type and spider1.qubit==spider2.qubit:
        spider1.phase+=spider2.phase #phaseaddition
        return spider1
```

The overall code achieved up to today:

```
import math
circuit=[("h",0),("CNOT",0,1),("Z",1,math.pi/2)]######
print(circuit)

class Spider:
    def __init__(self, spider_type, qubit, phase=0):#initialises the Spider - this is necessary
        self.type=spider_type #Z or X - building the language layer.
        self.qubit=qubit#added qubit index
        self.phase=phase
        self.connections=[]
    def __str__(self):#added for clean printing
        return f"{self.type}-spider (q{self.qubit},phase={self.phase})"#makes output more readable
def GateToSpider(gate):#second function and passing gate as a function.
    if gate[0]=="H":
        return [Spider("X",gate[1])]# gate[n] so that every spider knows which qubit it lives on.
    elif gate[0]=="Z":
        if len(gate)==3:#checks if user included a phase - if ys use it if no default to 0.#####
            return [Spider("Z",gate[1],gate[2])]####
        else:####
            return[Spider("Z",gate[1])]#####
    elif gate[0]=="CNOT":
        control=Spider("Z",gate[1])
        target=Spider("X",gate[2])
        #then connect the two
        control.connections.append(target)
        target.connections.append(control)
        return [control,target]#now the function returns a list of spiders, not just one.
    else:
        return []
def fuse(spider1, spider2):
    if spider1.type==spider2.type and spider1.qubit==spider2.qubit:
        spider1.phase+=spider2.phase #phaseaddition
        return spider1
CNOTtest=GateToSpider(("CNOT",0,1))
for spider in CNOTtest:#testingcode
    print(spider.type,"connected to",
          [s.type for s in spider.connections])
def ZXgraphbuild(circuit):#made a function
    graph=[]
    for gate in circuit:
        spiders=GateToSpider(gate)
```

```
for spider in CNOTtest:#testingcode
    print(spider.type,"connected to",
          [s.type for s in spider.connections])
def ZXgraphbuild(circuit):#made a function
    graph=[]
    for gate in circuit:
        spiders=GateToSpider(gate)
        graph.extend(spiders)
    return graph
zxgraph=ZXgraphbuild(circuit)
for spider in zxgraph:
    print(spider,"connected to",[f"{s.type}(q{s.qubit})" for s in spider.connections])
```

The current implementation shown above focuses on constructing a structural, internal representation of ZX graphs from quantum circuits. This backend design is vital to the project as it separates the mathematical representation from visual rendering.

The next stage of development is an interactive Tkinter based interface, where spiders are animated as the gates are applied, allowing users to see the transformation of qubit states in pictorial quantum reasoning, accessible to all.

The current implementation shown focuses on constructing a structured internal representation of ZX-graphs from quantum circuits. This backend design is separates the mathematical representation from visual rendering.

The next stage of development is an interactive Tkinter-based interface, where spiders are animated showing gates being applied. This will allow users to see the transformation from circuit notation to pictorial quantum reasoning in real time.

Week 5 – 22/04/26 – Today I plan to create a basic visual output of my ZX graph: for the spiders and connections. Not something that is interactive just yet but something that demonstrates the core concepts of ZX and my code.

Firstly, I am going to add positions to my spiders as they currently exist but have no position so Tkinter cannot draw them.

```
class Spider:
    self.x=0#add location x
    self.y=0#add location y
```

```
def assignpositions(graph):
    for i, spider in enumerate(graph):
        spider.x=100+i*100#assigning positions
        spider.y=200#
```

```
zxgraph=ZXgraphbuild(circuit)
assignpositions(zxgraph)#calling it
```

Next, I'm going to add my first minimal Tkinter window:

```
[('h', 0), ('CNOT', 0, 1), ('Z', 1, 1.5707963267948966)]
Traceback (most recent call last):
  File "C:\Users\21CarmignaniV\OneDrive - Lancaster Girls' Grammar School\persc
al\arkwright\week3operations.py", line 5, in <module>
    class Spider:
  File "C:\Users\21CarmignaniV\OneDrive - Lancaster Girls' Grammar School\persc
al\arkwright\week3operations.py", line 6, in Spider
    self.x=0#add location x
NameError: name 'self' is not defined
```

When I first ran the code it turned out I had accidentally put the self. commands outside where I had initialised the spider.

This is what the code now looks like:

```
import math
circuit=[("h",0),("CNOT",0,1),("Z",1,math.pi/2)]######
print(circuit)

class Spider:
    def __init__(self, spider_type, qubit, phase=0):#initialises the Spider - this is necessary
        self.type=spider_type #Z or X - building the language layer.
        self.qubit=qubit#added qubit index
        self.phase=phase
        self.connections=[]
        self.x=0#add location x
        self.y=0#add location y
    def __str__(self):#added for clean printing
        return f"{self.type}-spider (q{self.qubit},phase={self.phase})"#makes output more readable
def GateToSpider(gate):#second function and passing gate as a function.
    if gate[0]=="H":
        return [Spider("X",gate[1])]# gate[n] so that every spider knows which qubit it lives on.
    elif gate[0]=="Z":
        if len(gate)==3:#checks if user included a phase - if ys use it if no default to 0.#####
            return [Spider("Z",gate[1],gate[2])]####
        else:####
            return[Spider("Z",gate[1])]#####
    elif gate[0]=="CNOT":
        control=Spider("Z",gate[1])
        target=Spider("X",gate[2])
        #then connect the two
        control.connections.append(target)
        target.connections.append(control)
        return [control,target]#now the function returns a list of spiders, not just one.
    else:
        return []
def fuse(spider1, spider2):
    if spider1.type==spider2.type and spider1.qubit==spider2.qubit:
        spider1.phase+=spider2.phase #phaseaddition
        return spider1
CNOTtest=GateToSpider(("CNOT",0,1))
for spider in CNOTtest:#testingcode
    print(spider.type,"connected to",
          [s.type for s in spider.connections])
def ZXgraphbuild(circuit):#made a function
```

```
def ZXgraphbuild(circuit):#made a function
    graph=[]
    def assignpositions(graph):
        for i, spider in enumerate(graph):
            spider.x=100+i*100#assigning positions
            spider.y=200#
    for gate in circuit:
        spiders=GateToSpider(gate)
        graph.extend(spiders)
    return graph
zxgraph=ZXgraphbuild(circuit)
for spider in zxgraph:
    print(spider,"connected to",[f"{s.type}(q{s.qubit})" for s in spider.connections])
zxgraph=ZXgraphbuild(circuit)
assignpositions(zxgraph)#calling it
import tkinter as tk

def drawgraph(graph):
    root=tk.Tk()
    root.title("ZX graph demo")
    canvas=tk.Canvas(root,width=800,height=400)
    canvas.pack()
    #drawing the connections first
    for spider in graph:
        for conn in spider.connections:
            canvas.createline(spider.x,spider.y,conn.x,conn.y)
    #drawing spiders next
    for spider in graph:
        colour="green" if spider.type =="Z" else "red"
        canvas.createoval(spider.x-15,spider.y-15,
                          spider.x+15,spider.y+15,
                          fill=colour)
        canvas.createtext(spider.x,spider.y-25,
                          text=f"q{spider.qubit}")
    root.mainloop()
drawgraph(zxgraph)#running window
```

It then came up with this error message:

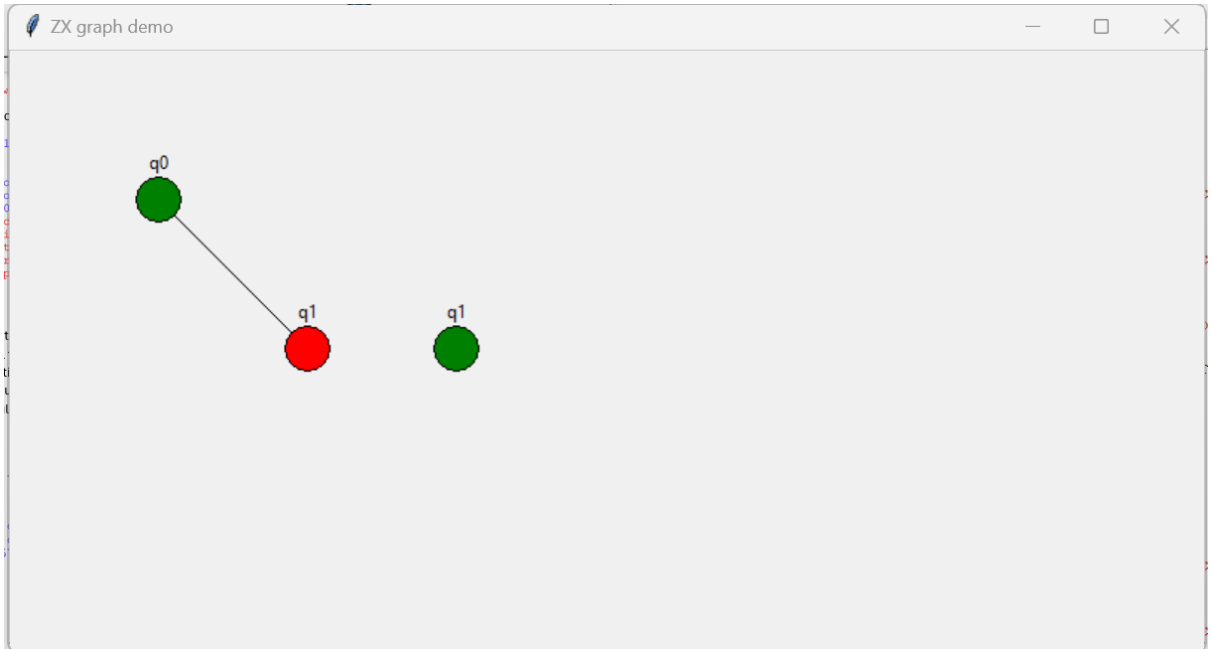
```
[('h', 0), ('CNOT', 0, 1), ('Z', 1, 1.5707963267948966)]
Z connected to ['X']
X connected to ['Z']
Z-spider (q0,phase=0) connected to ['X(q1)']
X-spider (q1,phase=0) connected to ['Z(q0)']
Z-spider (q1,phase=1.5707963267948966) connected to []
Traceback (most recent call last):
  File "C:\Users\21CarmignaniV\OneDrive - Lancaster Girls' Grammar School\person
al\arkwright\week3operations.py", line 54, in <module>
    assignpositions(zxgraph)#calling it
NameError: name 'assignpositions' is not defined
```

What I had done wrong was that I had defined assignpositions inside ZXgraphbuild which meant that it was treated as a local function as opposed to a global one. I then moved it outside so it is now a global function. Additionally, I noticed that if I left spider.y=200 then all the spiders would be on the same line but changing it to spider.y=100+spider.qubit*100 meant that q0 would be on top and q1 would be below, ensuring that the graph looks as much like a circuit as possible.

This was the python output:

```
[('h', 0), ('CNOT', 0, 1), ('Z', 1, 1.5707963267948966)]
Z connected to ['X']
X connected to ['Z']
Z-spider (q0,phase=0) connected to ['X(q1)']
X-spider (q1,phase=0) connected to ['Z(q0)']
Z-spider (q1,phase=1.5707963267948966) connected to []
```

And this was the tkinter output:



This is the final successful code from today's session fully re-commented:

```
import math
circuit=[("h",0),("CNOT",0,1),("Z",1,math.pi/2)]######
print(circuit)

class Spider:
    def __init__(self, spider_type, qubit, phase=0):
        #initialise a ZX-calculus "spider"(node in the graph).
        #type: "z" or "X" representing the basis.
        #qubit: which wire this spider belongs to.
        #phase: rotation(defaults to 0 if not specified).
        self.type=spider_type
        self.qubit=qubit
        self.phase=phase
        #stores connections to other spiders(edges in the graph).
        self.connections=[]
        #positions on screen(used later for visualisation.
        self.x=0
        self.y=0
    def __str__(self):#added for clean printing.
        return f"{self.type}-spider (q{self.qubit},phase={self.phase})"#makes output more readable
def GateToSpider(gate):
    #converts a quantum gate into one or more spiders.
    #this forms a 'translation layer' from circuit to ZX graph.
    if gate[0]=="H":
        # Hadamard approximated as an X-spider.
        return [Spider("X",gate[1])]
    elif gate[0]=="Z":
        #Z gate becomes a Z-spider,possibly with a phase.
        if len(gate)==3:
            return [Spider("Z",gate[1],gate[2])]
        else:
            return[Spider("Z",gate[1])]
    elif gate[0]=="CNOT":
        #CNOT creates two spiders(control and target).
        control=Spider("Z",gate[1])
        target=Spider("X",gate[2])
        #connect them - this represents entanglement.
        control.connections.append(target)
        target.connections.append(control)
        return [control,target]
    else:
```

```
        #unknown gate, return nothing as a safe fallback.
        return []
def fuse(spider1, spider2):
    if spider1.type==spider2.type and spider1.qubit==spider2.qubit:
        spider1.phase+=spider2.phase #phaseaddition.
        return spider1
CNOTtest=GateToSpider(("CNOT",0,1))
for spider in CNOTtest:#testingcode.
    print(spider.type,"connected to",
          [s.type for s in spider.connections])
def ZXgraphbuild(circuit):
    #builds the full ZX graph from a list of gates.
    #output - a list of spiders with connections.
    graph=[]
    for gate in circuit:
        spiders=GateToSpider(gate)
        graph.extend(spiders)#add all spiders to this graph.
    return graph
def assignpositions(graph):
    #assigns 2D coordinates to each spider for drawing.
    #x=progression through circuit(left to right).
    #y=qubit index (top to bottom).
    for i, spider in enumerate(graph):
        spider.x=100+i*100
        spider.y=100+spider.qubit*100
zxgraph=ZXgraphbuild(circuit)
for spider in zxgraph:
    print(spider,"connected to",[f"{s.type}(q{s.qubit})" for s in spider.connections])
zxgraph=ZXgraphbuild(circuit)
assignpositions(zxgraph)
import tkinter as tk

def drawgraph(graph):
    #creates a window and draws the ZX graph visually.
    root=tk.Tk()
    root.title("ZX graph demo")
    canvas=tk.Canvas(root,width=800,height=400)
    canvas.pack()
    #drawing the connections first so the lines apppear behind nodes.
    for spider in graph:
        for conn in spider.connections:
```

```
            canvas.create_line(spider.x,spider.y,conn.x,conn.y)
    #drawing spiders next(nodes)
    for spider in graph:
        #colour encodes spider type (Z=green,X=red)
        colour="green" if spider.type =="Z" else "red"
        canvas.create_oval(spider.x-15,spider.y-15,
                           spider.x+15,spider.y+15,
                           fill=colour)
        #label with qubit index
        canvas.create_text(spider.x,spider.y-25,
                           text=f"q{spider.qubit}")
    #kepp the window open
    root.mainloop()
drawgraph(zxgraph)
```

Week 6 - 27/04/26 - Today I really want to make progress towards an interactive feature but not jumping towards making it fully interactive in one go and breaking what I have currently working. So, instead, I am aiming to add one small interactive input feature that demonstrates the idea. I would like the user to be able to type in one gate and qubits, press a button, and see it drawn.

```
currentcircuit=[]#enables building the circuit live instead of a fixed circuit.
```

Next, I added input boxes in Tkinter UI.

```
gate_entry=tk.Entry(root)
gate_entry.pack()
qubit_entry=tk.Entry(root)
qubit_entry.pack()
info_label=tk.Label(root, text="Enter gate (H, Z, or CNOT) and qubit(s)")
info_label.pack()
```

Next, I added the function for the button that builds the circuit:

```
def addgate():
    gate=gate_entry.get().upper()
    qubits=qubit_entry.get()
    if gate=="CNOT":
        q0,q1=map(int, qubits.split(","))
    else:
        q=int(qubits)
        currentcircuit.append((gate,q))
    update_graph()
```

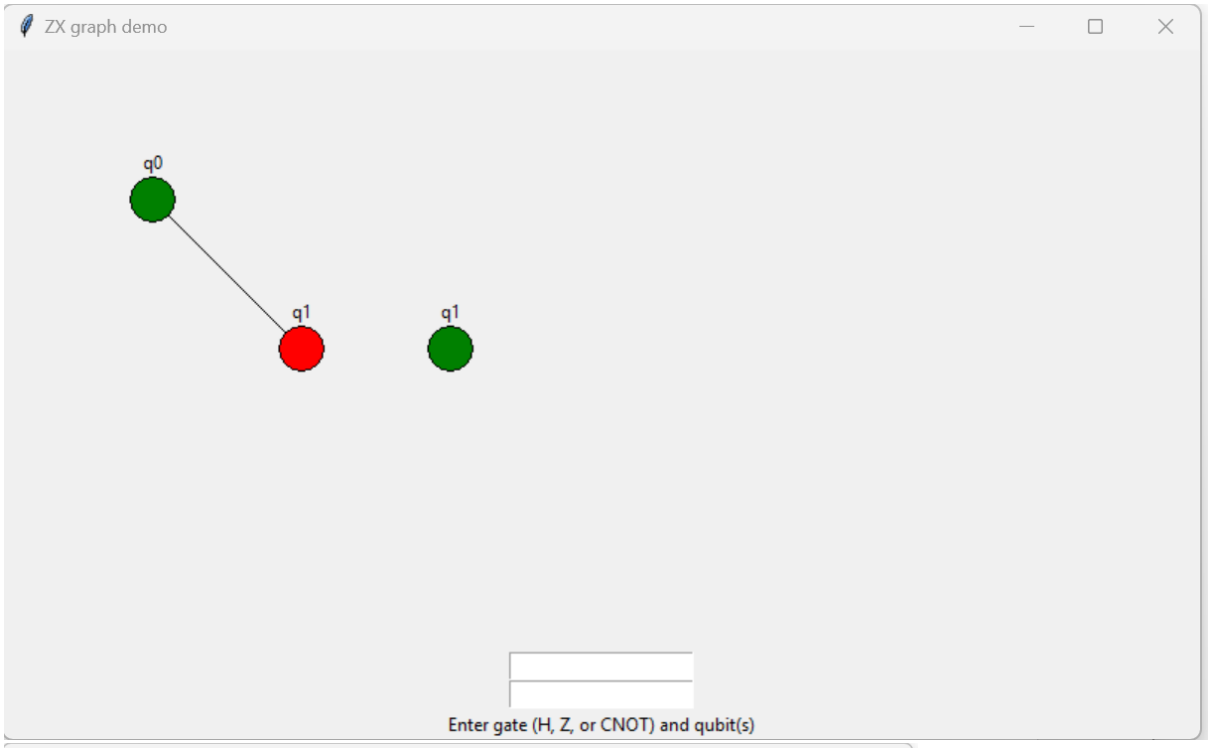
I also added a function to redraw the graph when the user clicks the button:

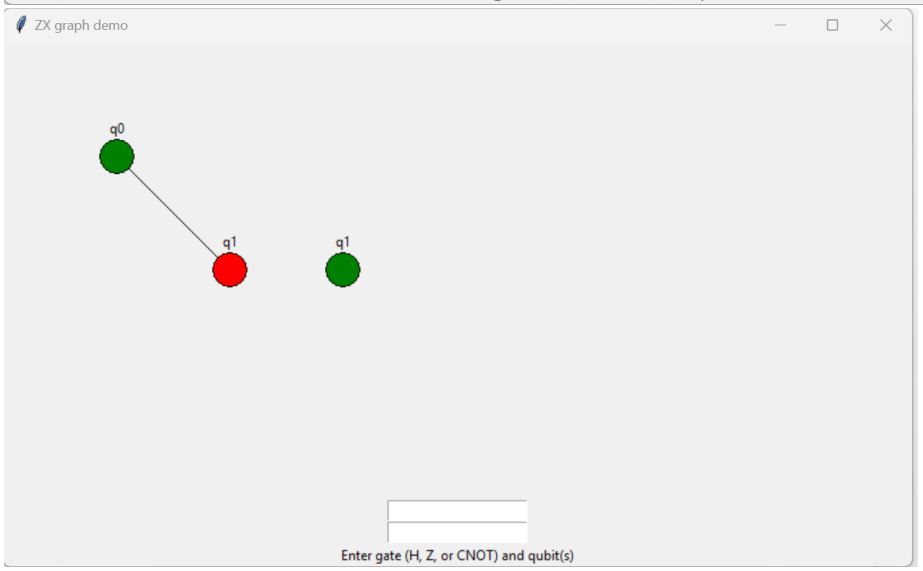
```
def updategraph():
    canvas.delete("all")#this clears the screen
    graph=ZXgraphbuild(currentcircuit)
    assignpositions(graph)
    #draw connections
    for spider in graph:
        for conn in spider.connections:
            canvas.create_line(spider.x, spider.y, conn.x, conn.y, width=2)
    #draw spiders
    for spider in graph:
        colour="green" if spider.type=="Z" else "red"
        canvas.create_oval(
            spider.x-15, spider.y-15,
            spider.x+15, spider.y+15,
            fill=colour
        )
        canvas.create_text(
            spider.x,
            spider.y-25,
            text=f"{spider.type}(q{spider.qubit})"
        )
```

And lastly for today, added the button:

```
addbutton=tk.Button(root,text="Add gate", command=addgate)
addbutton.pack()
```

I ran the code and it was not drawing the graph gradually and the button did not come up. This is what was showing:





I concluded that the functions addgate() and updategraph() are in the wrong scope and they need access to the canvas, currentcircuit and root. Currently, they're floating outside so they can't see those variables.

I am going to rewrite drawgraph() slightly and nest my functions inside of it:

```
def drawgraph(graph):
    #creates a window and draws the ZX graph visually.
    root=tk.Tk()
    root.title("ZX graph demo")
    canvas=tk.Canvas(root,width=800,height=400)
    canvas.pack()
    #drawing the connections first so the lines apppear behind nodes.
    for spider in graph:
        for conn in spider.connections:
            canvas.create_line(spider.x,spider.y,conn.x,conn.y)
    #drawing spiders next(nodes)
    for spider in graph:
        #colour encodes spider type (Z=green,X=red)
        colour="green" if spider.type =="Z" else "red"
        canvas.create_oval(spider.x-15,spider.y-15,
                           spider.x+15,spider.y+15,
                           fill=colour)
        #label with qubit index
        canvas.create_text(spider.x,spider.y-25,
                           text=f"q{spider.qubit}")
    #keep the window open
    gate_entry=tk.Entry(root)
    gate_entry.pack()
    qubit_entry=tk.Entry(root)
    qubit_entry.pack()
    info_label=tk.Label(root, text="Enter gate (H, Z, or CNOT) and qubit(s)")
    info_label.pack()
    def updategraph():
            canvas.delete("all")#this clears the screen
            graph=ZXgraphbuild(currentcircuit)
            assignpositions(graph)
            #draw connections
            for spider in graph:
                for conn in spider.connections:
                    canvas.create_line(spider.x, spider.y, conn.x, conn.y, width=2)
            #draw spiders
            for spider in graph:
                colour="green" if spider.type=="Z" else "red"
                canvas.create_oval(
                    spider.x-15, spider.y-15,
                    spider.x+15, spider.y+15,
                    fill=colour
                )
                canvas.create_text(
                    spider.x,
                    spider.y-25,
                    text=f"{spider.type}(q{spider.qubit})"
                )
    def addgate():
        gate=gate_entry.get().upper()
        qubits=qubit_entry.get()
        if gate=="CNOT":
            q0,q1=map(int, qubits.split(","))
        else:
            q=int(qubits)
            currentcircuit.append((gate,q))
        updategraph()
    addbutton=tk.Button(root,text="Add gate", command=addgate)
    addbutton.pack()
    root.mainloop()
drawgraph(zxgraph)
```

When I ran it, it did not come up with an error message, but the window was blank and nothing was being drawn so I concluded that something is either faulty with the updategraph() function or the button is not working:



First, I tested the button with print messages which did not appear which told me the error was in the button itself, so I amended the addgate() function:

```
def addgate():
    gate=gate_entry.get().upper()
    qubits=qubit_entry.get()
    if gate=="CNOT":
        try:
            q0,q1=map(int, qubits.replace(" "," ").split(","))
            currentcircuit.append((gate,q0,q1))
            updategraph()
        except:
            print("Please enter CNOT qubits like:0,1")
    else:
        q=int(qubits)
        currentcircuit.append((gate,q))
    updategraph()
```

The problem was that when the input was being split and the number has a space in front of it it was identifying it as " 1" which causes an error as it is not an integer. To fix this I removed the spaces and replaced any numbers inputted with spaces with the integer and wrapped the commands with input safety so that it doesn't crash.

This is the full commented code:

```
import math
currentcircuit=[]#stores the circuit as the user builds it interactively.
#ech entry is a gate tuple like ("H",0) or ("CNOT",0,1)
circuit=[("h",0),("CNOT",0,1),("Z",1,math.pi/2)]######
print(circuit)

class Spider:
    def __init__(self, spider_type, qubit, phase=0):
        #initialise a ZX-calculus "spider"(node in the graph).
        #type: "z" or "X" representing the basis.
        #qubit: which wire this spider belongs to.
        #phase: rotation(defaults to 0 if not specified).
        self.type=spider_type
        self.qubit=qubit
        self.phase=phase
        #stores connections to other spiders(edges in the graph).
        self.connections=[]
        #positions on screen(used later for visualisation.
        self.x=0
        self.y=0
    def __str__(self):#added for clean printing.
        return f"{self.type}-spider (q{self.qubit},phase={self.phase})"#makes output more readable
def GateToSpider(gate):
    #converts a quantum gate into one or more spiders.
    #this forms a 'translation layer' from circuit to ZX graph.
    if gate[0]=="H":
        # Hadamard approximated as an X-spider.
        return [Spider("X",gate[1])]
    elif gate[0]=="Z":
        #Z gate becomes a Z-spider,possibly with a phase.
        if len(gate)==3:
            return [Spider("Z",gate[1],gate[2])]
        else:
            return[Spider("Z",gate[1])]
    elif gate[0]=="CNOT":
        #CNOT creates two spiders(control and target).
        control=Spider("Z",gate[1])
        target=Spider("X",gate[2])
        #connect them - this represents entanglement.
        control.connections.append(target)
        target.connections.append(control)
```

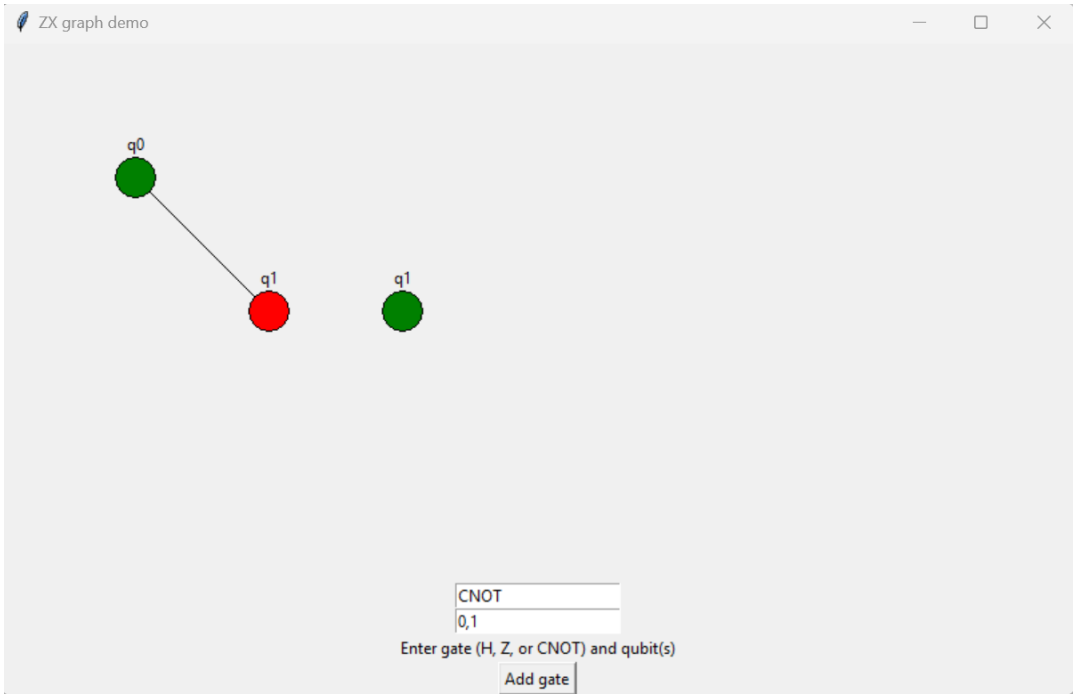
```
        return [control,target]
    else:
        #unknown gate, return nothing as a safe fallback.
        return []
def fuse(spider1, spider2):
    if spider1.type==spider2.type and spider1.qubit==spider2.qubit:
        spider1.phase+=spider2.phase #phaseaddition.
        return spider1
CNOTtest=GateToSpider(("CNOT",0,1))
for spider in CNOTtest:#testingcode.
    print(spider.type,"connected to",
          [s.type for s in spider.connections])
def ZXgraphbuild(circuit):
    #builds the full ZX graph from a list of gates.
    #output - a list of spiders with connections.
    graph=[]
    for gate in circuit:
        spiders=GateToSpider(gate)
        graph.extend(spiders)#add all spiders to this graph.
    return graph
def assignpositions(graph):
    #assigns 2D coordinates to each spider for drawing.
    #x=progression through circuit(left to right).
    #y=qubit index (top to bottom).
    for i, spider in enumerate(graph):
        spider.x=100+i*100
        spider.y=100+spider.qubit*100
zxgraph=ZXgraphbuild(circuit)
for spider in zxgraph:
    print(spider,"connected to",[f"{s.type}(q{s.qubit})" for s in spider.connections])
zxgraph=ZXgraphbuild(circuit)
assignpositions(zxgraph)
#Interactive visualisation - this section allows the user to build a quantum circuit live.
#Each gate entered is converted into a ZX graph and redrawn in real time.
#This demonstrates how abstract quantum operations map to graphical structures that are easier to understand.
import tkinter as tk
def drawgraph(graph):
    #creates a window and draws the ZX graph visually.
    root=tk.Tk()
    root.title("ZX graph demo")
    canvas=tk.Canvas(root,width=800,height=400)
```

```
    canvas.pack()
    #drawing the connections first so the lines apppear behind nodes.
    for spider in graph:
        for conn in spider.connections:
            canvas.create_line(spider.x,spider.y,conn.x,conn.y)
    #drawing spiders next(nodes)
    for spider in graph:
        #colour encodes spider type (Z=green,X=red)
        colour="green" if spider.type =="Z" else "red"
        canvas.create_oval(spider.x-15,spider.y-15,
                           spider.x+15,spider.y+15,
                           fill=colour)
        #label with qubit index
        canvas.create_text(spider.x,spider.y-25,
                           text=f"q{spider.qubit}")
    #keep the window open
    gate_entry=tk.Entry(root)
    gate_entry.pack()
    #input box where the user types the gate name (H,Z,CNOT).
    qubit_entry=tk.Entry(root)
    qubit_entry.pack()
    #input box where the user types the qubits.
    info_label=tk.Label(root, text="Enter gate (H, Z, or CNOT) and qubit(s)")
    info_label.pack()
    #instruction label to guide the user
    def updategraph():
            canvas.delete("all")#this clears the canvas so we can redraw from scratch.
            graph=ZXgraphbuild(currentcircuit)# rebuilds ZX graph from the updates circuit.
            assignpositions(graph)#assign screen coordinates to each spider.
            #draw connections
            for spider in graph:
                for conn in spider.connections:
                    canvas.create_line(spider.x, spider.y, conn.x, conn.y, width=2)
            #draw spiders
            for spider in graph:
                colour="green" if spider.type=="Z" else "red"#Z spiders are green and X spiders are red.
                canvas.create_oval(
                    spider.x-15, spider.y-15,
                    spider.x+15, spider.y+15,
                    fill=colour
                )#draw the circular node
                   canvas.create_text(
                       spider.x,
                       spider.y-25,
                       text=f"{spider.type}(q{spider.qubit})"
                   )#label the node with type and qubit index.
    def addgate():#read the user input from text boxes
        gate=gate_entry.get().upper()
        qubits=qubit_entry.get()
        if gate=="CNOT":
            try:#remove spaces and split input like "0,1" into two integers.
                q0,q1=map(int, qubits.replace(" ","").split(","))
                currentcircuit.append((gate,q0,q1))
                updategraph()#add a 2-qubit gate to the circuit
            except:#basic error handling for invalid input.
                print("Please enter CNOT qubits like:0,1")
        else:#single qubit gate
            q=int(qubits)
            currentcircuit.append((gate,q))
        updategraph()#redraw the graph to show the new circuit.
    addbutton=tk.Button(root,text="Add gate", command=addgate)
    addbutton.pack()#button that triggers addgate() when clicked.
    root.mainloop()#starts the Tkinter event loop (keeps the window open and responsive).
drawgraph(zxgraph)
```

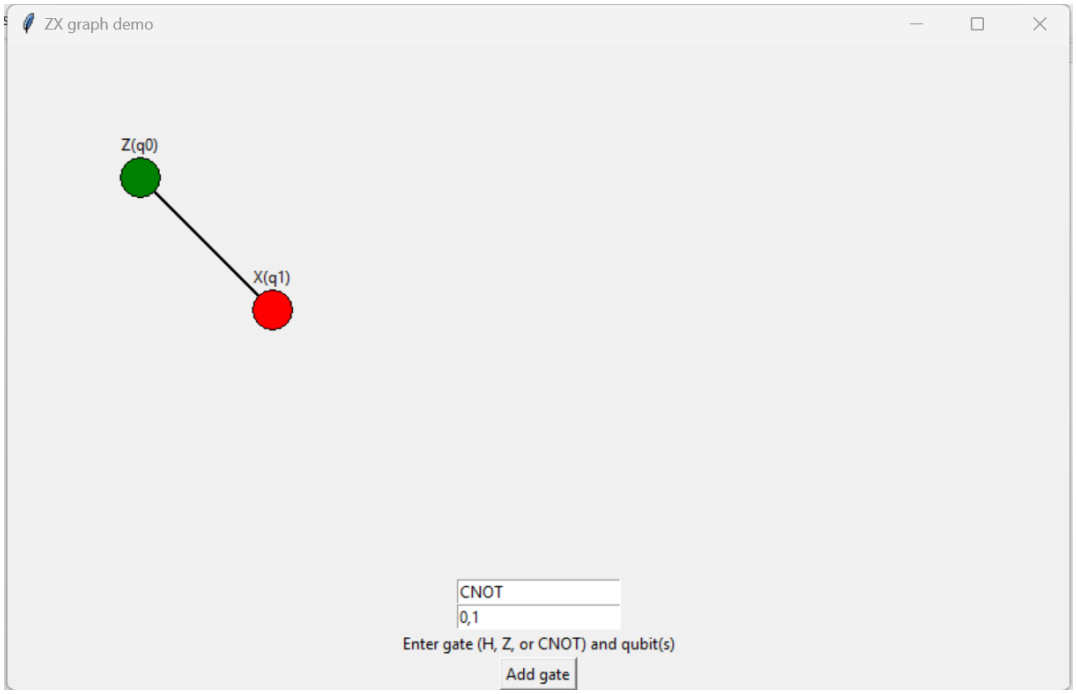
This is the result when the inputs are entered before the button is pressed:

> A CNOT is effectively a controlled NOT. It involves two qubits: the control qubit and the target qubit. The rule is:
>
> **‘If the control qubit is 1, flip the target qubit.’**

The green node (Z) represents the control cubit and the red node (X) represents the target cubit and the linne between them represents their interaction or the connection where information flows between them.



After the button is pressed:



05/06/26 – week 7 – Today I am wanting to ensure that my prototype can support the representation of one more gate – the Hadamard.

The simplified representation of this gate that I was previously using was not accurate as Hadamard includes two nodes and a connection as opposed to just one node so this alteration had to be made:

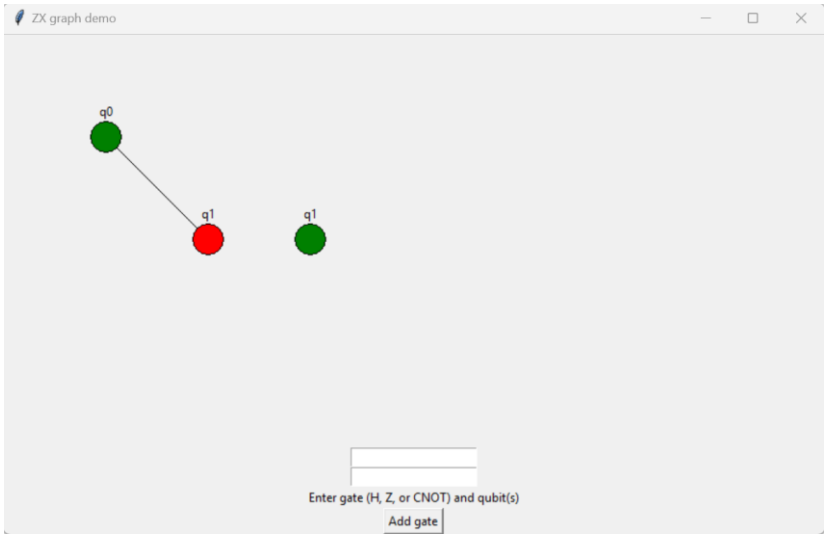
I deleted this:

```
if gate[0]=="H":
    # Hadamard approximated as an X-spider.
    return [Spider("X",gate[1])]
```

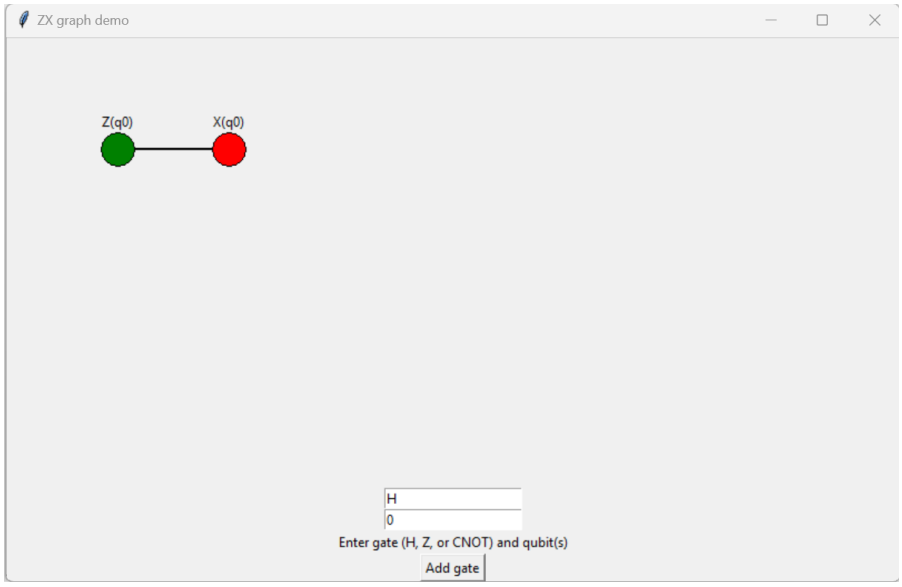
And changed it to this:

```
if gate[0] == "H":
    #create two spiders on the same qubit
    z=Spider("Z", gate[1])
    x=Spider("X", gate[1])
    #connect them (this is the hadamard)
    z.connections.append(x)
    x.connections.append(z)
    return[z,x]
```

When the code is run this is the Tkinter window:



Then when “H” and “0” are inputted in the boxes and add gate is pressed, this is the result:



The Hadamard gate puts a qubit into superposition and switches between binary (0/1) and Hadamard basis. In simple terms, it changes how we're looking at the qubit.

It is shown in my code as a Z and X node connected on the same qubit. This represents a change of basis, rather than an interaction between the two cubits unlike CNOT, shown by the fact that the qubits are on the same 'row' and opposed to on different 'rows' like on CNOT.

Next, I am going to attempt to animate the nodes and connections as opposed to have them just appear:

I am deleting this part of the drawing code:

```
            for spider in graph:
                for conn in spider.connections:
                    canvas.create_line(spider.x, spider.y, conn.x, conn.y, width=2)
            #draw spiders
            for spider in graph:
                colour="green" if spider.type=="Z" else "red"#Z spiders are green and X spiders are red.
                canvas.create_oval(
                    spider.x-15, spider.y-15,
                    spider.x+15, spider.y+15,
                    fill=colour
                )#draw the circular node
                canvas.create_text(
                    spider.x,
                    spider.y-25,
                    text=f"{spider.type}(q{spider.qubit})"
                )#label the node with type and qubit index.
```

This is the new code:

```
def updategraph():
        canvas.delete("all")#this clears the canvas so we can redraw from scratch.
        graph=ZXgraphbuild(currentcircuit)# rebuilds ZX graph from the updates circuit.
        assignpositions(graph)#assign screen coordinates to each spider.
        #draw connections
        for spider in graph:
            #draw connections for this spider
            for conn in spider.connections:
                canvas.create_line(
                    spider.x,spider.y,
                    conn.x,conn.y,
                    width=2
                )
        index=0 #which spider we're on.
        def drawstep():
            nonlocal index
            if index>= len(graph):
                return #stop when finished
            spider=graph[index]
            #draw the spider node
            colour="green" if spider.type == "Z" else "red"
            canvas.create_oval(
                spider.x-15, spider.y-15,
                spider.x+15, spider.y+15,
                fill=colour
            )
            #label it
            canvas.create_text(
                spider.x,
                spider.y-25,
                text=f"{spider.type}(q{spider.qubit})"
                )
            index+=1
            #schedule next step (300ms later)
            root.after(300, drawstep)
        drawstep()
```

Now it draws the same things but gradually.